

SOC 690S: Machine Learning in Causal Inference

Week 14: Feature Engineering for Causal Inference¹

Wenhao Jiang
Department of Sociology, Fall 2025



¹I heavily draw upon the materials from Chapter 10 of the textbook of Causal Machine Learning by Chernozhukov et al. (2026)

Introduction and Motivation

- Social scientists increasingly leverage unstructured, multimodal data to study human behavior, social interactions, and cultural and institutional processes (Lazer et al. 2009)
- These data are often represented as vector *embeddings* for machines to understand and analyze

Introduction and Motivation

- Social scientists increasingly leverage unstructured, multimodal data to study human behavior, social interactions, and cultural and institutional processes (Lazer et al. 2009)
- These data are often represented as vector *embeddings* for machines to understand and analyze
- Sociologists have primarily used embeddings in two ways:
 - As *objects of study* that capture cultural meaning, symbolic associations, and social representations (Kozlowski et al. 2019)
 - As predictive features that improve downstream tasks, often by fine-tuning pre-trained models (Bonikowski et al. 2022)

Introduction and Motivation

- Social scientists increasingly leverage unstructured, multimodal data to study human behavior, social interactions, and cultural and institutional processes (Lazer et al. 2009)
- These data are often represented as vector *embeddings* for machines to understand and analyze
- Sociologists have primarily used embeddings in two ways:
 - As *objects of study* that capture cultural meaning, symbolic associations, and social representations (Kozlowski et al. 2019)
 - As predictive features that improve downstream tasks, often by fine-tuning pre-trained models (Bonikowski et al. 2022)
- Despite their substantive richness, embeddings are rarely incorporated as *features* into standard causal inference, even though they may encode important confounding information that should be adjusted for in regression analyses

Introduction and Motivation

- Causal inference researchers are interested in estimating the treatment effect of D on the outcome Y

$$Y = \beta_1 D + \beta_2' W + \epsilon$$

- In observational settings, we rely on the *strong ignorability* assumption, which requires that all confounders are observed and properly modeled by $\beta_2' W$

Introduction and Motivation

- Causal inference researchers are interested in estimating the treatment effect of D on the outcome Y

$$Y = \beta_1 D + \beta_2' W + \epsilon$$

- In observational settings, we rely on the *strong ignorability* assumption, which requires that all confounders are observed and properly modeled by $\beta_2' W$
- This is a strong assumption because there are numerous potential confounders, some of which are difficult to measure using, for example, survey data

Introduction and Motivation

- Consider the *neighborhood effects* literature, where neighborhood context is argued to have a causal effect on individual outcomes
- For example, in studying the effect of gunshot presence in a neighborhood on children's academic performance, one may need to control for overall neighborhood “quality”
- Increasingly, researchers use data such as images and text to measure neighborhood “quality,” which is often not captured in survey data

Introduction and Motivation

- Consider estimating the *price elasticity of demand*: the causal effect of price on consumer demand
- Comparing products that differ substantially in quality, design, or target market can confound this relationship
- Credible estimation requires controlling for product quality and other characteristics, but these attributes are often difficult to measure using conventional structured data (Bajari et al. 2025)

Introduction and Motivation

- Consider estimating the *price elasticity of demand*: the causal effect of price on consumer demand
- Comparing products that differ substantially in quality, design, or target market can confound this relationship
- Credible estimation requires controlling for product quality and other characteristics, but these attributes are often difficult to measure using conventional structured data (Bajari et al. 2025)
 - However, they may be encoded in product images or text as embeddings

Overview of the Talk

- Introduce several approaches for transforming complex data—particularly text, which encodes rich information—into low-dimensional representations
- Review Double Machine Learning (DML), which enables flexible adjustment for high-dimensional feature representations in regression²
 - Brief review of OLS, the Frisch–Waugh–Lovell (FWL) theorem, and their asymptotic properties
 - Introduction to *Neyman Orthogonality*
- Illustrate the methods using the example of estimating the price elasticity of demand from Amazon's data

²“Low-dimensional” is relative to the original text data, which may contain millions of unique words and their relationships; “high-dimensional” is relative to the sample size ($p/n \not\rightarrow 0$)

Double Machine Learning (DML)

OLS and the Frisch–Waugh–Lovell (FWL) Theorem

- Consider the linear regression model under low-dimensional regime

$$Y = \beta_1 D + \beta_2' W + \epsilon$$

- FWL Theorem states that the coefficient β_1 can be obtained by regressing the residualized outcome on the residualized treatment

$$\beta_1 = \arg \min_b E[(\tilde{Y} - b\tilde{D})^2]$$

$$\tilde{D} = D - \gamma_D' W; \quad \gamma_D = \arg \min_{\gamma} E[(D - \gamma' W)^2]$$

$$\tilde{Y} = Y - \gamma_Y' W; \quad \gamma_Y = \arg \min_{\gamma} E[(Y - \gamma' W)^2]$$

²For a proof, see p. 32 week 1 slides: <https://github.com/wenhaojiangsoc/CML/tree/main>

OLS and the Frisch–Waugh–Lovell (FWL) Theorem

- Under regularity conditions and low-dimensional data $p/n \rightarrow 0$

$$\sqrt{n}(\hat{\beta}_1 - \beta_1) \xrightarrow{d} \mathcal{N}\left(0, E[\tilde{D}\tilde{D}']^{-1} E[\epsilon^2 \tilde{D}\tilde{D}'] E[\tilde{D}\tilde{D}']^{-1}\right)$$

- $\hat{\beta}_1$ is $\sqrt{n}$ -consistent
- The consistent “sandwich” estimator (Eicker-Huber-White) of a *sample* is then given by

$$\hat{V}(\hat{\beta}_1) = (\tilde{D}\tilde{D}')^{-1} \left(\sum_i^n \tilde{D}\tilde{D}'\hat{\epsilon}^2 \right) (\tilde{D}\tilde{D}')^{-1}$$

by plugging in sample $\hat{\epsilon}^2$ to estimate ϵ^2

Double Machine Learning (DML)

- Under high-dimensional regime $p/n \not\rightarrow 0$, the nuisance regressions can no longer be estimated precisely enough for conventional OLS inference; $\sqrt{n}$ asymptotic normality generally fails
- Double Machine Learning (DML) yields $\sqrt{n}$ -consistent estimates and valid standard errors under *Neyman Orthogonality* and cross-fitting

Double Machine Learning (DML)

- Consider a partially linear model under a high-dimensional regime

$$Y = \beta_1 D + \eta(W) + \epsilon$$

$$\beta_1 = \arg \min_b E[(\tilde{Y} - b\tilde{D})^2]$$

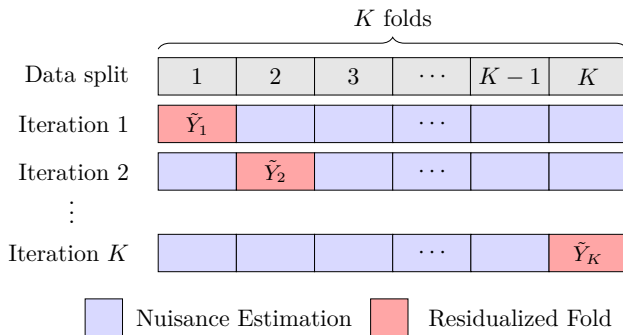
$$\tilde{Y} = Y - E[Y | W]$$

$$\tilde{D} = D - E[D | W]$$

- The nuisance functions $g(W) = E[Y | W]$ and $m(W) = E[D | W]$ can be estimated flexibly using nonlinear models.
- Machine learning provides a natural class of estimators for these nuisance functions when W is high-dimensional.

Cross-Fitting in Double Machine Learning (DML)

- Nuisance functions and residualization are operationalized through *cross-fitting*



- The nuisance functions $g(W) = E[Y | W]$ and $m(W) = E[D | W]$ are estimated in the blue fold, and $\tilde{Y}$ and $\tilde{D}$ are estimated in the red fold

Neyman Orthogonality in Double Machine Learning (DML)

- The key property underlying Double Machine Learning is *Neyman Orthogonality* through *cross-fitting*
- Consider the moment function

$$\begin{aligned}\psi(Y, D, W; \beta, g, m) &= (\tilde{Y} - \beta \tilde{D}) \tilde{D} \\ \tilde{Y} &= Y - g(W), \quad \tilde{D} = D - m(W)\end{aligned}$$

Neyman Orthogonality in Double Machine Learning (DML)

- The key property underlying Double Machine Learning is *Neyman Orthogonality* through *cross-fitting*
- Consider the moment function

$$\begin{aligned}\psi(Y, D, W; \beta, g, m) &= (\tilde{Y} - \beta \tilde{D}) \tilde{D} \\ \tilde{Y} &= Y - g(W), \quad \tilde{D} = D - m(W)\end{aligned}$$

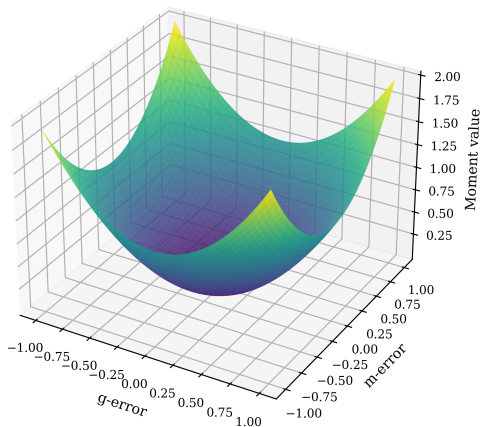
- The moment is Neyman orthogonal if its (Gateaux) derivative with respect to the nuisance functions vanishes at the truth

$$\left. \frac{\partial E[\psi(Y, D, W; \beta, g, m)]}{\partial(g, m)} \right|_{(g, m) = (g_0, m_0)} = 0$$

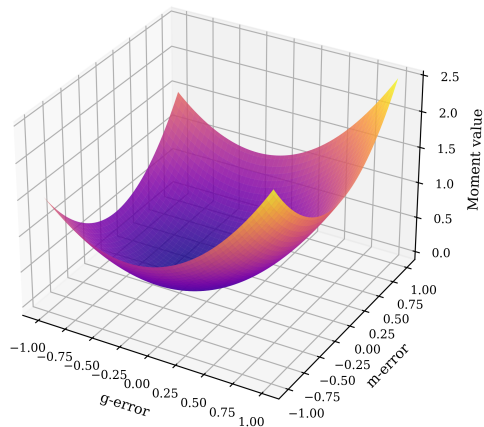
- Intuitively, the moment function is locally *flat* in the nuisance direction
- Consequently, small errors in estimating g and m contribute only *second-order* bias, allowing valid $\sqrt{n}$ -inference for β_1

Double Machine Learning: Neyman Orthogonality

Orthogonal Moment Surface



Non-Orthogonal Moment Surface



Double Machine Learning with Embeddings

- We estimate the effect of product price on consumer demand while controlling for high-dimensional covariates W , such as text or image embeddings that encode rich information about product characteristics
- Rather than assuming a fully linear model, we consider the partially linear model

$$Y = \beta_1 D + \eta(W) + \epsilon$$

- The target parameter β_1 is estimated by

$$\hat{\beta}_1 = \arg \min_b E[(\tilde{Y} - b\tilde{D})^2]$$

where $\tilde{Y}$ and $\tilde{D}$ are residualized using flexible machine learning methods with cross-fitting

Feature Extraction

Feature Extraction: Principal Components

- We are interested in transforming complex objects such as text into a collection of relatively low-dimensional numerical features
- Principal components are an early classic example of embeddings
- One way to frame principal components is that principal components find orthonormal linear combinations that are “best” at reproducing the underlying data

Feature Extraction: Principal Components

- We are interested in transforming complex objects such as text into a collection of relatively low-dimensional numerical features
- Principal components are an early classic example of embeddings
- One way to frame principal components is that principal components find orthonormal linear combinations that are “best” at reproducing the underlying data
- A small number of principal components should capture most of the variability in the original variables and provide a useful low-dimensional summary of the original data

Feature Extraction: Principal Components

- Let $(W_1, \dots, W_n)$ be a sample of n observations of a high-dimensional centered random vector W in $\mathbb{R}^d$, and let $\Sigma_n = \mathbb{E}[WW'] \in \mathbb{R}^{d \times d}$ denote the empirical covariance matrix

Feature Extraction: Principal Components

- Let $(W_1, \dots, W_n)$ be a sample of n observations of a high-dimensional centered random vector W in $\mathbb{R}^d$, and let $\Sigma_n = \mathbb{E}[WW'] \in \mathbb{R}^{d \times d}$ denote the empirical covariance matrix
- The eigenvector c_k in $\mathbb{R}^d$ and eigenvalue λ_k is found by

$$\Sigma_n c_k = \lambda_k c_k$$

- We find the K ($K \ll d$) largest eigenvalues in an descending order and corresponding eigenvectors $C_K = [c_1, \dots, c_K]$
- We find W_i 's projection on each eigenvector (principal direction)

$$X_{ki} = c_k' W_i, \quad k = 1, \dots, K$$

- Principal components $X_i^K \in \mathbb{R}^K$

$$X_i^K = (X_{1i}, \dots, X_{Ki})'$$

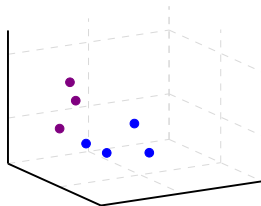
Feature Extraction: Principal Components

- Principal components $X_i^K \in \mathbb{R}^K$

$$X_i^K = (X_{1i}, \dots, X_{Ki})'$$

- X_i^K represents the compressed feature vector of the original high-dimensional vector W

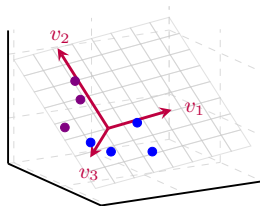
Original embedding space



PCA/SVD



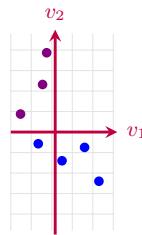
Principal directions



Project



2D PC projection



Feature Extraction: Principal Components

- Principal components $X_i^K \in \mathbb{R}^K$

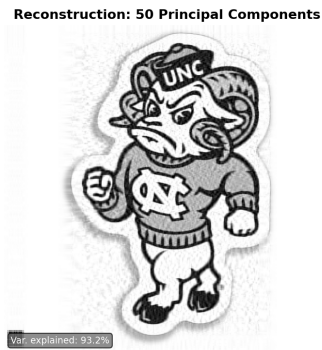
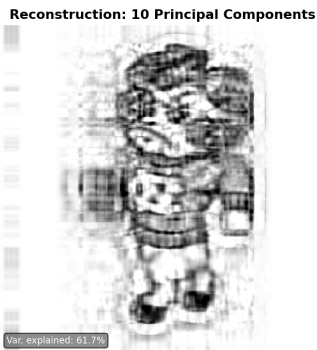
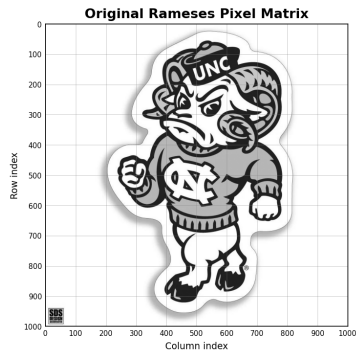
$$X_i^K = (X_{1i}, \dots, X_{Ki})'$$

- X_i^K represents the compressed feature vector of the original high-dimensional vector W
- One common use of Principal Components is to compress image
- We approximate the original vector W_i by projecting X_i^K back to the d -dimensional space

$$\hat{W}_i = C_K X_i^K$$

- The choice of K determines how much of the original information (variance) in W_i is retained

Principal Components as Image Compression



Principal Components in Other Applications

- Principal Component Analysis (PCA), or equivalently Singular Value Decomposition (SVD), underlies many important applications
- Latent Semantic Analysis (LSA) creates low-dimensional embeddings for words and documents from a word–document matrix
- Recommender systems create low-dimensional embeddings for users and items from a user–item interaction matrix

Principal Components Projection as Autoencoder

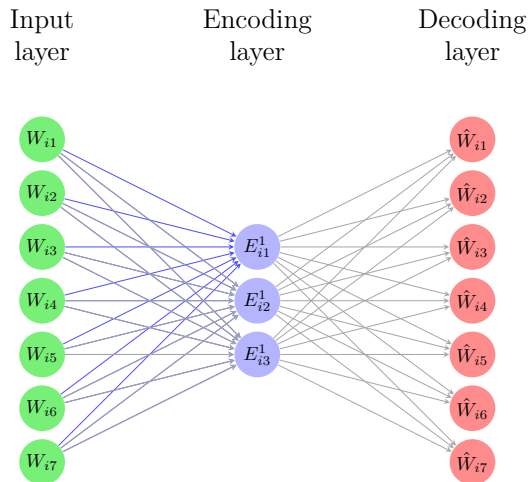
- The approximation to W_i from W can be seen as a predictive exercise
- It underlies a shallow one-layer *linear* neural network

$$W_i \mapsto C'_K W_i := E_i \mapsto A' E_i := \hat{W}_i$$

$d \times 1$ $K \times 1$ $d \times 1$

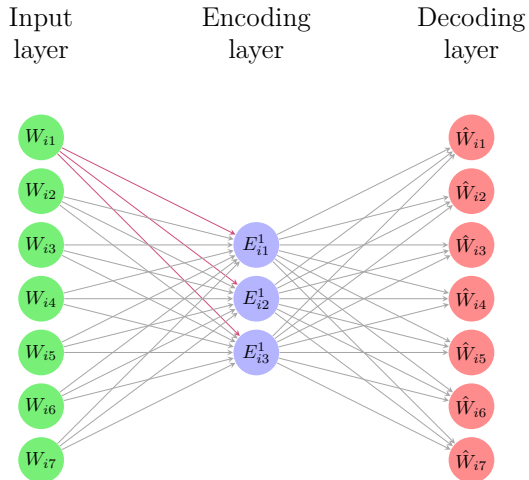
- The first step is said to be “encoding” the information in the input, and the second step is said to be “decoding” in the sense of returning the encoded information to the original space
- Principal components projections are embeddings generated by a linear “encoder–decoder” network (an *autoencoder*)
- For principal components projection, $A = C'_K$

Principal Components as Autoencoder

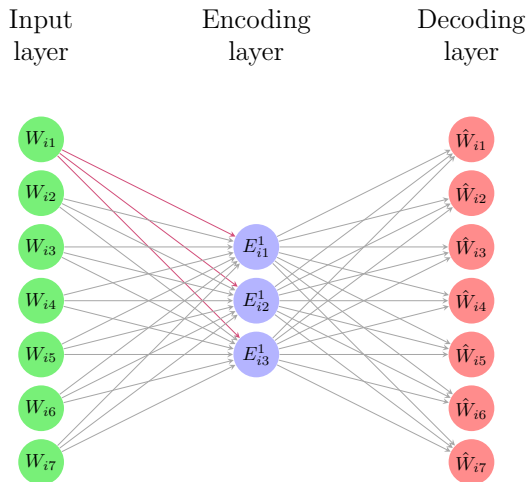


- The weights connecting the input to each hidden neuron correspond to a principal direction (eigenvector)
- The encoding layer outputs are the principal component
- The decoder reconstructs the input using these components

Principal Components as Autoencoder

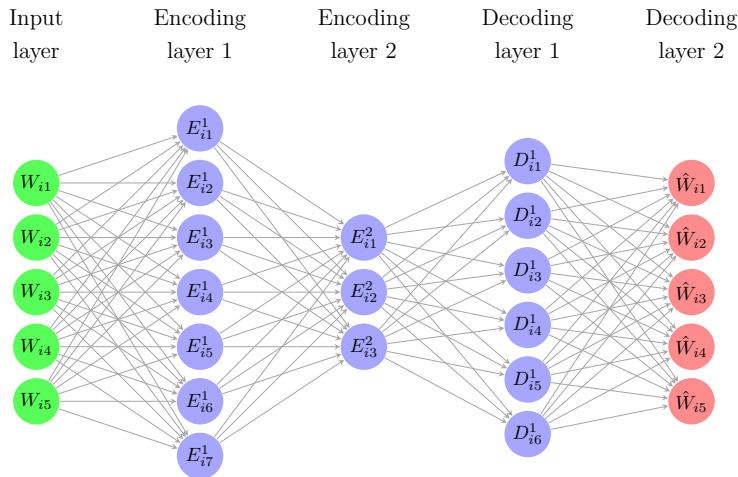


Principal Components as Autoencoder



- The weights connecting each input neuron to hidden neurons correspond to each feature's weight or coefficient on each latent dimension

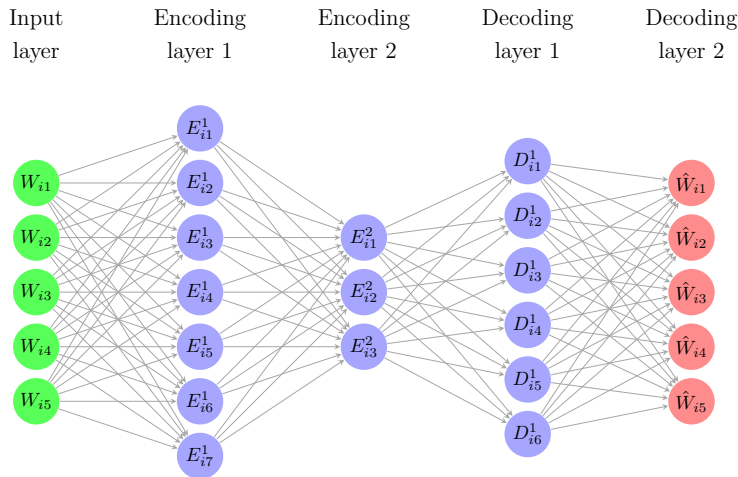
From Principal Components To Autoencoder



- The approach can be generalized to nonlinear encoders and decoders with multiple layers with *nonlinear* activation functions, forming deep autoencoders

$$W_i \xrightarrow{g^1} E_i^1 \dots \xrightarrow{g^k} E_i^k \xrightarrow{g^{k+1}} D_i^{k+1} \dots \xrightarrow{g^m} D_i^m := \hat{W}_i$$

From Principal Components To Autoencoder



- Deep autoencoders preserve more information in the original data
- The trained encoder (the weights) is used to generate embeddings for downstream tasks

$$W_i \xrightarrow{g_1} E_i^1 \dots \xrightarrow{g_k} E_i^k \xrightarrow{g_{k+1}} D_i^{k+1} \dots \xrightarrow{g_m} D_i^m := \hat{W}_i$$

From Autoencoders to General Embeddings

- The encoder–decoder paradigm underlies many modern representation learning methods, including deep autoencoders and encoder–decoder Transformers such as T5 and BART
- In a classical autoencoder, the objective is to minimize the expected reconstruction loss

$$\mathbb{E}[\text{loss}(W, d(e(W)))]$$

where $e(\cdot)$ and $d(\cdot)$ denote the encoder and decoder

- More generally, the prediction target need not be the original input

$$\mathbb{E}[\text{loss}(A, f(e(W)))]$$

where A is an arbitrary learning target

From Autoencoders to General Embeddings

- More generally, the prediction target need not be the original input

$$\mathbb{E}[\text{loss}(A, f(e(W)))]$$

where A is an arbitrary learning target

- For example, BERT predicts masked words in a sentence, while T5 and BART reconstruct corrupted text from its encoded representation
- The resulting embeddings often capture more useful semantic information than simple reconstruction
- In our price elasticity example from Amazon products we will see later, we may prefer embeddings that encode product type and functionality rather than faithfully reconstruct the product description

Word2Vec

Word2Vec Embeddings

- Different learning objectives A produce different types of embeddings
 - Word2Vec learns embeddings by predicting a word from its surrounding context (or the other way around)
 - This idea is motivated by the *distributional hypothesis*: words are understood by their semantic context
-
- A bottle of **tesgüino** is on the table
 - Everyone likes **tesgüino**
 - **Tesgüino** makes you drunk
 - We make **tesgüino** out of corn

Word2Vec Embeddings

- Different learning objectives A produce different types of embeddings
- Word2Vec learns embeddings by predicting a word from its surrounding context (or the other way around)
- This idea is motivated by the *distributional hypothesis*: words are understood by their semantic context

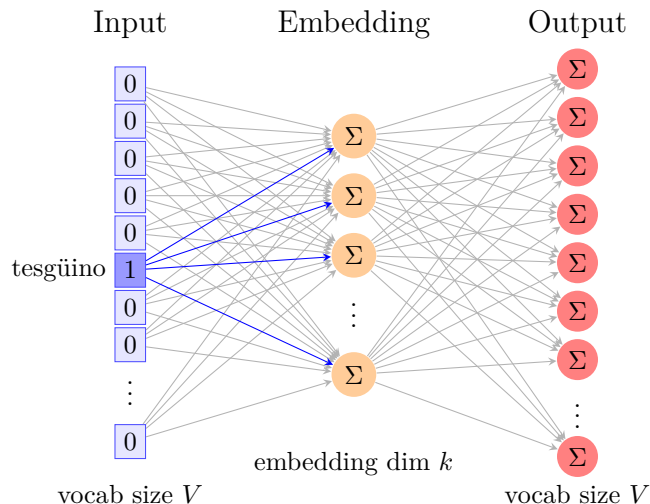
- A bottle of **tesgüino** is on the table
- Everyone likes **tesgüino**
- **Tesgüino** makes you drunk
- We make **tesgüino** out of corn



Word2Vec Embeddings

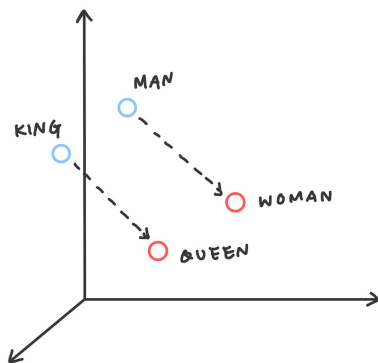
- Two training objectives are commonly used in Word2Vec
 - *Continuous Bag-of-Words (CBOW)*: predict the center word from its surrounding context
 - *Skip-gram*: predict the surrounding context words from the center word

Word2Vec: Skip-gram with Negative Sampling



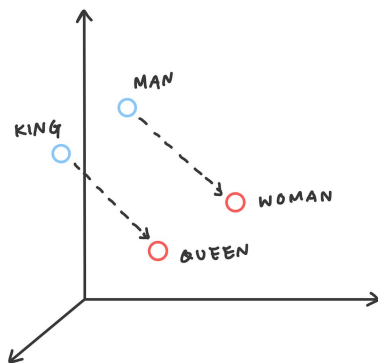
- Analogous to Principal Components Projection, each embedding “weight” (from input to embedding layer) learns the coefficient of the word to each latent semantic coordinate from the word co-occurrence structure

Word2Vec Embeddings



- Word2Vec learns a low-dimensional vector representation for each word
- Words appearing in similar contexts tend to have similar embeddings
- The learned embeddings capture semantic relationships, such as word analogies

Word2Vec Embeddings



- Word2Vec learns a low-dimensional vector representation for each word
- Words appearing in similar contexts tend to have similar embeddings
- The learned embeddings capture semantic relationships, such as word analogies
- However, the embeddings are context-independent: “bank” in “river bank” and “financial bank” receive the same representation
- Word order and syntax are not explicitly modeled; sentence embeddings formed by averaging word embeddings are often imprecise

ELMo

Sequence Models

- A major advance beyond Word2Vec was to represent text as a *sequence*, allowing embeddings to depend on word order and surrounding context
- For example, “CASER belongs to NYU Shanghai” conveys a different meaning than “NYU Shanghai belongs to CASER”

Sequence Models

- A major advance beyond Word2Vec was to represent text as a *sequence*, allowing embeddings to depend on word order and surrounding context
- For example, “CASER belongs to NYU Shanghai” conveys a different meaning than “NYU Shanghai belongs to CASER”
- One influential approach is *Embeddings from Language Models* (ELMo), which learns context-dependent word embeddings
- More generally, language models are inspired by the Shannon game: predict a missing or future word given the surrounding words in a sentence

Sequence Model: ELMo

- Let T_k denote the k -th token in a sentence of length n , and let t denote a candidate word in the vocabulary
- The forward language model predicts the next word using the preceding words

$$p_k^f(t) = P(T_{k+1} = t \mid T_1, \dots, T_k; \theta)$$

- The backward language model predicts the previous word using the following words:

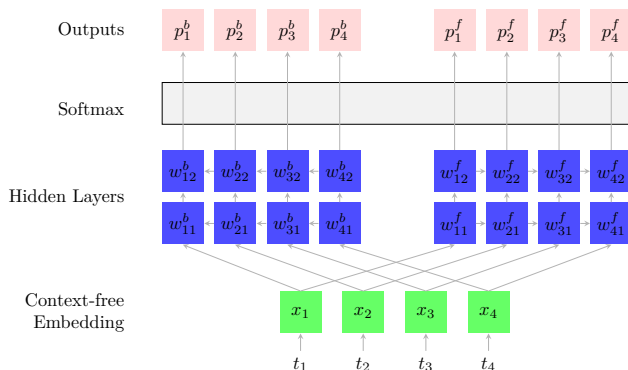
$$p_k^b(t) = P(T_{k-1} = t \mid T_k, \dots, T_n; \theta)$$

- Here θ denotes all model parameters

Sequence Model: ELMo

- Many social and economic processes are inherently sequential; for example, life-course trajectories in sociology and financial time series depend on past histories
- Likewise, language exhibits sequential dependence: the probability of the next word depends on the preceding words
- Recurrent Neural Networks (RNNs) model such sequential dependencies by maintaining a hidden state that evolves over the sequence
- To mitigate vanishing and exploding gradients over long sequences, ELMo employs Long Short-Term Memory (LSTM) networks, whose gated architecture enables information to persist across many time steps

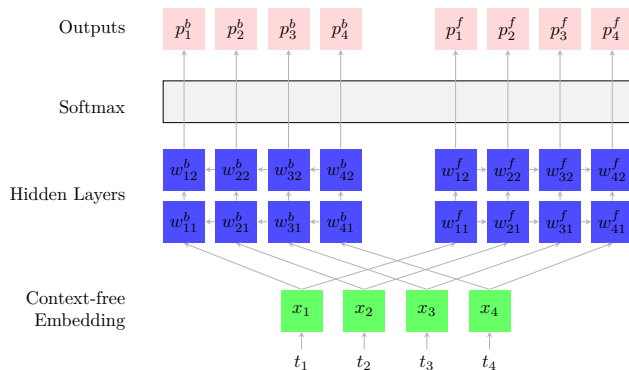
Sequence Model: ELMo



- Input x_i consists of context-free word embeddings
- Two LSTM networks are used, forward and backward
- After pre-training, each word is associated with $2L$ contextual hidden states, where L is the number of stacked LSTM layers in each direction

Sequence Model: ELMo: ELMo and Downstream Task

$$t_k \mapsto w_k := \sum_{i=1}^L (\gamma_i w_{ki}^f + \bar{\gamma}_i w_{ki}^b)$$



- $2L$ hidden states are combined to produce the final context-dependent ELMo embedding
- The weights γ and $\bar{\gamma}$ are learned during the downstream task (e.g., classification) to combine hidden states into context-dependent word representations

BERT

Transformer Architecture

- A recent advance in language modeling is the transformer architecture
- Rather than processing text sequentially, transformers use *self-attention* to model the relationships among all words in a sentence simultaneously
- Compared with LSTMs, self-attention more effectively captures long-range dependencies by allowing each word to attend directly to the most relevant words, regardless of their distance in the sequence

Transformer Architecture

- A recent advance in language modeling is the transformer architecture
- Rather than processing text sequentially, transformers use *self-attention* to model the relationships among all words in a sentence simultaneously
- Compared with LSTMs, self-attention more effectively captures long-range dependencies by allowing each word to attend directly to the most relevant words, regardless of their distance in the sequence
- For example,

*The Summer Institute in Computational Social Science (SICSS) at NYU Shanghai was hosted by CASER. **It** attracts smart audience from around the world.*

- Self-attention helps determine that “*it*” refers to “SICSS”, even though the relevant words are far apart

Transformer Architecture

- A prominent transformer-based language model is *Bidirectional Encoder Representations from Transformers* (BERT)
- The original BERT model is pre-trained using two self-supervised objectives
 - *Masked Language Modeling (MLM)*: Randomly mask a subset of words in a sentence and predict the masked words from their surrounding context
 - *Next Sentence Prediction (NSP)*: Given a pair of sentences, predict whether the second sentence immediately follows the first in the original text

Transformer Architecture: BERT³

- For a sentence of n tokens, each token is represented by the sum of its token embedding (e.g., Word2Vec) and positional embedding. Collecting all token representations gives

$$X \in \mathbb{R}^{n \times d}$$

- Query, Key, and Value matrices are computed by

$$Q = XW^Q \in \mathbb{R}^{n \times d_k}$$

$$K = XW^K \in \mathbb{R}^{n \times d_k}$$

$$V = XW^V \in \mathbb{R}^{n \times d_v}$$

$$W^Q, W^K \in \mathbb{R}^{d \times d_k}$$

$$W^V \in \mathbb{R}^{d \times d_v}$$

³Check out this site to calculate attention by hand

<https://www.byhand.ai/p/11-can-you-calculate-self-attention>

Transformer Architecture: BERT

- Self-attention is computed as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK'}{\sqrt{d_k}}\right) V$$

- Since $QK' \in \mathbb{R}^{n \times n}$, each token computes an attention score for every token in the sequence, including itself
- The resulting $n \times n$ attention matrix determines how information is aggregated across all tokens to produce contextual embeddings
 - Each entry asks how much should token i attend to token j
- The rows of $\text{Attention}(Q, K, V) \in \mathbb{R}^{n \times d_v}$ are contextual representations of the tokens, which are further refined by the remainder of the Transformer block

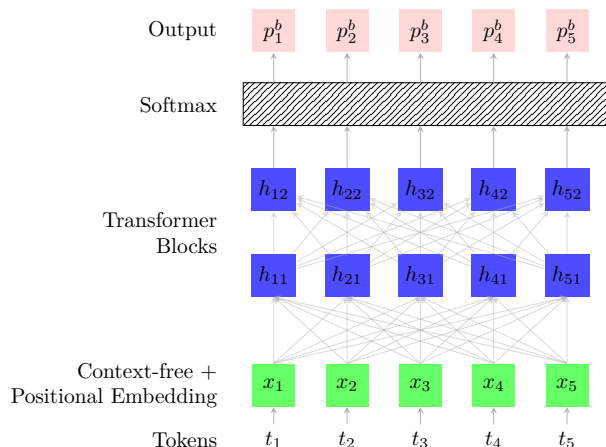
Transformer Architecture: BERT

- During pre-training, the projection matrices W^Q , W^K , and W^V (together with all other model parameters) are updated by gradient descent, and W^Q , W^K , and W^V are global model parameters
- For example, consider the masked sentence

[CLS] The SICSS at NYU Shanghai was hosted by [MASK] . [SEP]

- The model computes a contextual embedding for every token, uses the contextual embedding of [MASK] to predict the missing word, and compares the prediction with the true word (e.g., CASER)
- The prediction error is backpropagated to update W^Q , W^K , W^V , the feed-forward layers, and the token embeddings
- After training, the encoder (e.g., projection matrices, token embedding, residual connections) is retained to generate contextual embeddings

Transformer Architecture: BERT



- The last Transformer layer produces the final hidden states $h_i^{(L)}$, which serve as the contextual embedding for each token
- A sentence embedding can then be obtained by aggregating the token embeddings (e.g., averaging or pooling) or, in BERT, by using the contextual embedding of the special [CLS] token

Fine-Tuning BERT for Downstream Tasks

- There are two common approaches to adapting a pre-trained BERT model for downstream tasks
- *Linear probing*: Freeze the pre-trained BERT encoder and train only a linear prediction head on top of the final hidden states
- *Fine-tuning*: Update both the BERT encoder and the task-specific prediction head jointly using the downstream training data

Fine-Tuning BERT for Nuisance Function Estimation

- Consider the partially linear model under high-dimensional regime

$$Y = \beta_1 D + \eta(W) + \epsilon$$

$$\beta_1 = \arg \min_b E[(\tilde{Y} - b\tilde{D})^2]$$

$$\tilde{Y} = Y - E[Y \mid W]$$

$$\tilde{D} = D - E[D \mid W]$$

- Here, Y is product demand, D is price, and W is the BERT representation of the product description, obtained from the contextual embedding of the [CLS] token

Fine-Tuning BERT for Nuisance Function Estimation

- To better approximate the nuisance functions

$$g(W) = E[Y \mid W], \quad m(W) = E[D \mid W]$$

we adapt the pre-trained BERT model either by *linear probing* or by *fine-tuning*, which jointly updates both the BERT encoder and the prediction heads

- We fine-tune BERT to learn product representations that better predict both price and demand, thereby more effectively adjusting for confounding information contained in product descriptions
- Finally, we estimate price elasticity by applying Double Machine Learning to the residualized outcome $\tilde{Y}$ and treatment $\tilde{D}$ using cross-fitting

Thank You

- Thank you! Please feel free to reach out with questions or to chat
wenhao.jiang@duke.edu

Thank You

- Thank you! Please feel free to reach out with questions or to chat wenhao.jiang@duke.edu

I am also looking for volunteers to help pre-test a research project that Siwei Cheng (NYU) and I are conducting on social interactions at conferences

We do not collect your name, speech content, or mobility trajectory, only which badges you are close to



- If you are interested, please:
 - Open the web app, add it to your home screen, collect a badge, input your badge number in your app, and wear it throughout the afternoon and Happy Hour
 - At the end, return the badge to me or Cindy and confirm the return in the app

References

- Bajari, Patrick, Zhihao Cen, Victor Chernozhukov, Manoj Manukonda, Suhas Vijaykumar, Jin Wang, Ramon Huerta, Junbo Li, Ling Leng, George Monokroussos, and Shan Wang. 2025. “Hedonic prices and quality adjusted price indices powered by AI.” *Journal of Econometrics* 251:105652.
- Bonikowski, Bart, Yuchen Luo, and Oscar Stuhler. 2022. “Politics as Usual? Measuring Populism, Nationalism, and Authoritarianism in U.S. Presidential Campaigns (1952–2020) with Neural Language Models.” *Sociological Methods & Research* 51:1721–1787.
- Kozlowski, Austin C., Matt Taddy, and James A. Evans. 2019. “The Geometry of Culture: Analyzing the Meanings of Class through Word Embeddings.” *American Sociological Review* 84:905–949.
- Lazer, David, Alex Pentland, Lada Adamic, Sinan Aral, Albert-László Barabási, Devon Brewer, Nicholas Christakis, Noshir Contractor, James Fowler, Myron Gutmann, Tony Jebara, Gary King, Michael Macy, Deb Roy, and Marshall Van Alstyne. 2009. “Computational social science.” *Science* 323:721–723.